

Exercícios Resolvidos - Lista 2 - Análise de Algoritmos e Caracterização de Tempos de Execução

Disciplina: PCC104 - Projeto de Análise de Algoritmos

• Exercício 2

- (a) **Tamanho da entrada.** O tamanho da entrada é n , o número de elementos do arranjo A . O valor x e cada $A[i]$ ocupam $\Theta(1)$ espaço no modelo RAM usual (palavras de tamanho fixo), de modo que o tamanho dominante é o comprimento do arranjo.
- (b) **Tabela de custos.** Seja t o número de vezes que o teste do **while** na linha 2 é avaliado. Como o teste é avaliado uma vez extra (aquela que falha e encerra o laço), a linha 3 (incremento) executa exatamente $t - 1$ vezes.

Linha	Instrução	Custo	Vezez
1	$i \leftarrow 1$	c_1	1
2	while $i \leq n$ and $A[i] \neq x$	c_2	t
3	$i \leftarrow i + 1$	c_3	$t - 1$
4	if $i \leq n$	c_4	1
5	return i	c_5	0 ou 1
7	return -1	c_7	0 ou 1

Exatamente uma das linhas 5 ou 7 executa; denotemos por c_{ret} esse custo terminal (constante, independente de n).

- (c) **Expressão de $T(n)$.** Somando custo $\times$ frequência:

$$T(n) = c_1 + c_2 t + c_3 (t - 1) + c_4 + c_{\text{ret}} = (c_2 + c_3) t + \underbrace{(c_1 + c_4 + c_{\text{ret}} - c_3)}_{\text{constante}}.$$

- (d) **Melhor caso.** Ocorre quando o teste falha já na primeira iteração porque $A[1] = x$. Nesse caso, o **while** avalia o teste uma única vez (e sai), logo $t = 1$.

Instância: qualquer arranjo com $A[1] = x$, por exemplo $A = \langle x, *, *, \dots, * \rangle$.

Substituindo $t = 1$:

$$T_{\text{melhor}}(n) = (c_2 + c_3) \cdot 1 + (c_1 + c_4 + c_{\text{ret}} - c_3) = c_1 + c_2 + c_4 + c_{\text{ret}},$$

que é uma constante. Logo $T_{\text{melhor}}(n) = \Theta(1)$, com constantes $c_1 = c_2 = c_1 + c_2 + c_4 + c_{\text{ret}}$ e $n_0 = 1$ (na definição $c_1 \leq T_{\text{melhor}}(n) \leq c_2$ para todo $n \geq n_0$).

- (e) **Pior caso.** Ocorre quando o laço percorre todas as posições sem nunca encontrar x : o teste avalia para $i = 1, 2, \dots, n$ (verdadeiro) e mais uma vez para $i = n + 1$ (falso), totalizando $t = n + 1$.

Instâncias: todos os arranjos A tais que $A[i] \neq x$ para todo $i \in \{1, \dots, n\}$, isto é, $x \notin A$.

Substituindo $t = n + 1$:

$$T_{\text{pior}}(n) = (c_2 + c_3)(n + 1) + (c_1 + c_4 + c_{\text{ret}} - c_3) = (c_2 + c_3)n + \kappa,$$

com κ constante. Tomando $a = c_2 + c_3 > 0$, vale $an \leq T_{\text{pior}}(n) \leq 2an$ para $n \geq n_0$ suficientemente grande (basta $n_0 \geq |\kappa|/a$), portanto $T_{\text{pior}}(n) = \Theta(n)$.

- (f) **Caso médio.** Seja X a variável aleatória “posição da única ocorrência de x em A ”, uniforme em $\{1, 2, \dots, n\}$. Se $X = k$, o teste do **while** é avaliado k vezes: $k - 1$ vezes com resultado verdadeiro (linhas $A[i] \neq x$ para $i < k$) e uma vez com resultado falso em $i = k$. Logo $t \mid (X = k) = k$.

$$\mathbb{E}[t] = \sum_{k=1}^n k \cdot \Pr[X = k] = \frac{1}{n} \sum_{k=1}^n k = \frac{1}{n} \cdot \frac{n(n+1)}{2} = \frac{n+1}{2}.$$

Por linearidade da esperança aplicada a $T(n) = (c_2 + c_3)t + \kappa'$:

$$T_{\text{med}}(n) = \mathbb{E}[T(n)] = (c_2 + c_3) \frac{n+1}{2} + \kappa' = \frac{c_2 + c_3}{2} n + \kappa'',$$

com κ'' constante. Concluímos que $T_{\text{med}}(n) = \Theta(n)$.

Observação: o caso médio cresce na mesma classe do pior caso, mas com constante multiplicativa exatamente a metade.

• Exercício 3

- (a) **Tamanho da entrada.** O tamanho da entrada é n , o número de elementos do arranjo A .
- (b) **Pior caso: contagem do laço interno.** Para cada valor de $i \in \{1, \dots, n-1\}$, o laço interno itera j de $i+1$ até n , ou seja, $n-i$ iterações; em cada uma delas o teste da linha 2 é avaliado uma vez (a avaliação extra de saída custa apenas o teste falso da condição do **for**, que aqui contabilizamos junto). No pior caso, o algoritmo nunca retorna na linha 4, percorrendo todos os pares.

O número total de avaliações da comparação interna é

$$\sum_{i=1}^{n-1} (n-i) = \frac{n(n-1)}{2}.$$

Instância: qualquer arranjo com todos os elementos distintos, por exemplo $A = \langle 1, 2, 3, \dots, n \rangle$.

- (c) **Tabela de custos — pior caso.** Seja $S = \frac{n(n-1)}{2}$ o número de pares (i, j) com $i < j$.

Linha	Instrução	Custo	Vezez
1	for $i \leftarrow 1$ to $n-1$	c_1	n
2	for $j \leftarrow i+1$ to n	c_2	$\sum_{i=1}^{n-1} (n-i+1) = S + (n-1)$
3	if $A[i] = A[j]$	c_3	S
4	return verdadeiro	c_4	0
5	return falso	c_5	1

Observação: para o laço interno na linha 2, cada bloco de i produz $(n-i)$ testes verdadeiros mais 1 teste falso (que encerra o laço), totalizando $n-i+1$ avaliações; somando em i obtém-se $S + (n-1)$. A linha 3 (comparação) é executada apenas dentro do laço, S vezes.

Portanto,

$$T_{\text{pior}}(n) = c_1 n + c_2 (S + (n-1)) + c_3 S + c_5,$$

e substituindo $S = \frac{n(n-1)}{2}$:

$$T_{\text{pior}}(n) = \frac{c_2 + c_3}{2} n^2 + \left(c_1 + \frac{c_2 - c_3}{2} \right) n + (c_5 - c_2).$$

- (d) $T_{\text{pior}}(n) = \Theta(n^2)$. Escrevemos $T_{\text{pior}}(n) = a n^2 + b n + d$ com

$$a = \frac{c_2 + c_3}{2} > 0, \quad b = c_1 + \frac{c_2 - c_3}{2}, \quad d = c_5 - c_2.$$

Tome as constantes $\alpha_1 = a/2$, $\alpha_2 = 2a$ e n_0 grande o suficiente para que

$$\alpha_1 n^2 \leq a n^2 + b n + d \leq \alpha_2 n^2 \quad \text{para todo } n \geq n_0.$$

Para a cota superior, basta n_0 tal que $b n + d \leq a n^2$ a partir dele (válido pois o termo quadrático domina). Para a cota inferior, basta n_0 tal que $b n + d \geq -\frac{a}{2} n^2$ a partir dele. Ambos os n_0 são finitos, logo $T_{\text{pior}}(n) = \Theta(n^2)$.

- (e) **Melhor caso.** Ocorre quando $A[1] = A[2]$: na primeira iteração do laço externo, com $i = 1$ e $j = 2$, a linha 3 executa uma única vez, o teste é verdadeiro e o algoritmo retorna na linha 4.

Número de execuções da linha 3: exatamente 1. Cada uma das demais linhas executa $O(1)$ vezes. Portanto

$$T_{\text{melhor}}(n) = \Theta(1).$$

- (f) **Comparação com INSERTION-SORT.** Tanto TEM-DUPPLICATA quanto INSERTION-SORT apresentam $T_{\text{pior}}(n) = \Theta(n^2)$. A semelhança é estrutural: ambos os algoritmos varrem todos (ou quase todos) os pares de índices (i, j) com $i < j$, o que naturalmente leva à soma $\sum_{i=1}^{n-1} (n-i) = \frac{n(n-1)}{2}$. A diferença está no melhor caso: INSERTION-SORT sobre um arranjo já ordenado executa apenas a comparação inicial de cada iteração externa, atingindo $\Theta(n)$; TEM-DUPPLICATA, por sua vez, sai imediatamente quando o primeiro par $(1, 2)$ já é igual, atingindo $\Theta(1)$.

• Exercício 10

- (a) **Passos do paradigma D&C.**

- **Dividir:** calcular $q = \lfloor (p+r)/2 \rfloor$ (linha 4) particiona $A[p..r]$ em dois subarranjos contíguos $A[p..q]$ e $A[q+1..r]$, de tamanhos $\lceil n/2 \rceil$ e $\lfloor n/2 \rfloor$. Custa $\Theta(1)$.
- **Conquistar:** as chamadas recursivas das linhas 5 e 6 resolvem os dois subproblemas independentemente, retornando os máximos m_1 e m_2 . O caso-base é $p = r$ (linhas 1–3), em que o máximo é o próprio elemento $A[p]$.

- **Combinar:** a comparação $m_1 \geq m_2$ e a devolução do maior valor (linhas 7–11) combinam as duas soluções parciais em $\Theta(1)$.
- (b) **Relação de recorrência.** Cada chamada não trivial dispara 2 chamadas recursivas, cada uma sobre um subarranjo de tamanho aproximadamente $n/2$; o trabalho fora das chamadas recursivas (divisão + combinação) é $\Theta(1)$. O caso-base $n = 1$ executa apenas o teste da linha 1 e o retorno da linha 2, custando $\Theta(1)$. Logo,

$$T(n) = \begin{cases} \Theta(1), & \text{se } n = 1, \\ 2T(n/2) + \Theta(1), & \text{se } n > 1. \end{cases}$$

Para simplificar a álgebra, escrevemos $T(n) = 2T(n/2) + c$ com $c > 0$ constante e $T(1) = c$.

- (c) **Árvore de recursão, $n = 2^k$.** A cada nível, cada subproblema gera 2 filhos com metade do tamanho:

Nível ℓ	Nós	Tamanho de cada subproblema	Custo do nível
0	1	n	c
1	2	$n/2$	$2c$
2	4	$n/4$	$4c$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
ℓ	2^ℓ	$n/2^\ell$	$2^\ell c$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
$k = \log_2 n$	$2^k = n$	1	$n \cdot c$

A árvore tem $k + 1 = \log_2 n + 1$ níveis. O custo total é a soma sobre todos os níveis:

$$T(n) = \sum_{\ell=0}^k 2^\ell c = c(2^{k+1} - 1) = c(2n - 1).$$

Note que, ao contrário do que acontece no MERGE-SORT, aqui o custo por nível *não* é constante: ele dobra a cada nível porque $f(n) = \Theta(1)$ e o número de subproblemas cresce geometricamente. Em consequência, o custo é dominado pelas folhas.

- (d) $T(n) = \Theta(n)$. De $T(n) = c(2n - 1) = 2cn - c$, tomemos $\alpha_1 = c$ e $\alpha_2 = 2c$. Para todo $n \geq 1$ vale

$$\alpha_1 n = cn \leq 2cn - c \leq 2cn = \alpha_2 n,$$

pois $cn \leq 2cn - c \iff c \leq cn$, válido para $n \geq 1$. Portanto $T(n) = \Theta(n)$.

- (e) **Teorema Mestre.** Comparando $T(n) = 2T(n/2) + \Theta(1)$ com a forma canônica $T(n) = aT(n/b) + f(n)$:

$$a = 2, \quad b = 2, \quad f(n) = \Theta(1).$$

O expoente crítico é $\log_b a = \log_2 2 = 1$, logo $n^{\log_b a} = n$. Comparando $f(n)$ com $n^{\log_b a}$:

$$f(n) = \Theta(1) = O(n^{1-\varepsilon}) \quad \text{para qualquer } 0 < \varepsilon \leq 1.$$

Estamos no **Caso 1** do Teorema Mestre, que conclui

$$T(n) = \Theta(n^{\log_b a}) = \Theta(n),$$

confirmando o resultado obtido pela árvore de recursão.

- (f) **Comparação com a versão iterativa.** A varredura linear — inicializar $\max \leftarrow A[1]$ e percorrer $A[2..n]$ atualizando $\max$ quando $A[i] > \max$ — também é $\Theta(n)$. As duas abordagens estão na mesma classe assintótica.

Há, porém, uma diferença nas constantes e no consumo de memória. A versão recursiva executa $2n - 1$ chamadas no total (n folhas + $n - 1$ nós internos) e usa pilha de profundidade $\Theta(\log n)$; a iterativa faz exatamente $n - 1$ comparações com $\Theta(1)$ de espaço auxiliar.

A lição é que divisão e conquista *não traz ganho assintótico* quando o passo de combinação é $\Theta(1)$ e o problema original já admite solução linear: nesse cenário, $T(n) = 2T(n/2) + \Theta(1)$ cai no Caso 1 do Mestre e o custo é dominado pelas folhas, recuperando o mesmo $\Theta(n)$ da varredura. O paradigma só compensa quando combinar é mais barato que resolver do zero — como em MERGE-SORT, onde combinar custa $\Theta(n)$ mas resolver iterativamente custaria $\Theta(n^2)$ (INSERTION-SORT).

• Exercício 9

(a) **Tamanho dos subproblemas.**

Seja $n = r - p + 1$ o tamanho do subarranjo corrente e $q = \lfloor (p + r)/2 \rfloor$. As duas chamadas recursivas operam sobre os subarranjos $A[p..q-1]$ e $A[q+1..r]$, cujos tamanhos são, respectivamente,

$$n_{\text{esq}} = (q - 1) - p + 1 = q - p, \quad n_{\text{dir}} = r - (q + 1) + 1 = r - q.$$

Substituindo $q = \lfloor (p + r)/2 \rfloor$:

$$n_{\text{esq}} = \left\lfloor \frac{p+r}{2} \right\rfloor - p = \left\lfloor \frac{r-p}{2} \right\rfloor = \left\lfloor \frac{n-1}{2} \right\rfloor \leq \left\lfloor \frac{n}{2} \right\rfloor,$$

$$n_{\text{dir}} = r - \left\lfloor \frac{p+r}{2} \right\rfloor = \left\lceil \frac{r-p}{2} \right\rceil = \left\lceil \frac{n-1}{2} \right\rceil \leq \left\lceil \frac{n}{2} \right\rceil.$$

A última desigualdade vale porque $\lceil (n-1)/2 \rceil = \lfloor n/2 \rfloor$ para todo $n \geq 1$. Assim, qualquer que seja o ramo escolhido, o subproblema tem tamanho *no máximo* $\lfloor n/2 \rfloor$.

(b) **Recorrência do pior caso.**

O pior caso ocorre quando $x \notin A$ ou quando x se encontra em uma posição que exige descer até um subarranjo unitário. Em cada chamada, o trabalho não recursivo (cálculo de q , comparações nas linhas 1, 4 e 6 e retorno) é $\Theta(1)$. Há *uma* chamada recursiva sobre um subproblema de tamanho no máximo $\lfloor n/2 \rfloor$. O caso base ocorre quando $p > r$, isto é, $n = 0$, com custo constante. Portanto,

$$T(n) = \begin{cases} \Theta(1), & \text{se } n \leq 1, \\ T(\lfloor n/2 \rfloor) + \Theta(1), & \text{se } n \geq 2. \end{cases}$$

(c) **Resolução por iteração para $n = 2^k$.**

Para $n = 2^k$, os pisos são exatos: $\lfloor n/2 \rfloor = n/2 = 2^{k-1}$. Seja c uma constante positiva que limita o custo não recursivo. Então:

$$\begin{aligned} T(n) &= T(n/2) + c \\ &= [T(n/4) + c] + c = T(n/4) + 2c \\ &= [T(n/8) + c] + 2c = T(n/8) + 3c \\ &\vdots \\ &= T(n/2^i) + i c. \end{aligned}$$

A iteração termina quando $n/2^i = 1$, ou seja, $i = \log_2 n = k$. Logo,

$$T(n) = T(1) + k c = \Theta(1) + c \log_2 n.$$

Tomando $T(1) = d$ constante, obtemos a forma fechada

$$\boxed{T(n) = c \log_2 n + d.}$$

(d) **Classe assintótica:** $T(n) = \Theta(\log n)$.

Da expressão fechada $T(n) = c \log_2 n + d$, exibimos constantes $c_1, c_2 > 0$ e n_0 tais que

$$c_1 \log_2 n \leq T(n) \leq c_2 \log_2 n, \quad \forall n \geq n_0.$$

Basta tomar, por exemplo, $n_0 = 2$, $c_1 = c$ (pois $d \geq 0$) e $c_2 = c + d$ (pois $\log_2 n \geq 1$ para $n \geq 2$, donde $d \leq d \log_2 n$). Portanto $T(n) = \Theta(\log_2 n) = \Theta(\log n)$, já que logaritmos em bases distintas diferem apenas por fator constante.

(e) **Melhor caso.**

O melhor caso ocorre quando $A[q] = x$ na *primeira* chamada, isto é, quando o elemento procurado encontra-se exatamente na posição mediana inicial. Nesse cenário não há recursão e o algoritmo executa apenas trabalho constante:

$$T_{\text{melhor}}(n) = \Theta(1).$$

(f) **Aplicação do Teorema Mestre.**

A recorrência do pior caso tem a forma $T(n) = aT(n/b) + f(n)$, com

$$a = 1, \quad b = 2, \quad f(n) = \Theta(1).$$

Calculamos o expoente crítico:

$$\log_b a = \log_2 1 = 0, \quad n^{\log_b a} = n^0 = 1.$$

Comparando $f(n) = \Theta(1)$ com $n^{\log_b a} = \Theta(1)$, vemos que ambos são polinomialmente equivalentes, ou seja, $f(n) = \Theta(n^{\log_b a})$. Aplica-se, portanto, o **Caso 2** do Teorema Mestre, que fornece

$$T(n) = \Theta\left(n^{\log_b a} \log n\right) = \Theta(\log n),$$

confirmando o resultado obtido pelo método da iteração.